Day 60 - Performance and Scalability Report

Benchmark findings, service changes, and scaling plan | 26 October 2026

Summary. Vectorized skill comparisons reduced warm scoring time from 92.1 ms to 36.8 ms per resume in the eight-resume benchmark, about 2.5x faster. Service-level concurrency testing showed that more threads do not automatically increase throughput. No HTTP API exists in the repository, so results below are local scoring measurements rather than API response measurements.

Service changes

- Skill embeddings are collected once per input list and pairwise cosine scores are calculated as a matrix in **scoring/semantic_matching_engine.py**.
- **CandidateMatchingService.generate_scores()** now accepts an iterable of resumes and returns scores in input order. It reuses the process-level model and cache, but scores each resume sequentially; it does not batch transformer inference.

Warm scoring benchmark

The original run scored eight resumes in 0.7371 s total (92.1 ms average). After vectorization and the cosine-matrix refinement, the latest run scored them in 0.2943 s total (36.8 ms average). That is a 60% reduction in average warm scoring time. The first model load and warm-up are excluded from this comparison.

Three displayed scores shifted slightly from the original run: CV ASIF, 52.36% to 51.97%; cv data, 69.59% to 69.32%; and cv-2, 69.88% to 69.78%. The other five displayed scores were unchanged. Review these small differences against expected scoring behavior before deployment.

Memory and cache

The LRU embedding cache is already bounded at 256 entries. The eight-resume workload populated 51 entries; at 384 float32 values per embedding, the maximum vector payload is approximately 0.38 MB. A process-memory run reached a 482 MB working set after scoring eight resumes. This is a single local observation, not a production capacity guarantee. The existing tracemalloc benchmark starts after model initialization and therefore does not include model/native-runtime memory.

Concurrent load simulation

The simulation used one warmed process, eight labeled resumes, and three rounds per concurrency level (24 requests per level). The cache was cleared between rounds. It does not include HTTP, multiple processes, or a load balancer; percentile values come from a small sample.

- Default PyTorch threading: concurrency 1 = 12.72 requests/s, 78.40 ms mean, 303.38 ms P95; concurrency 2 = 11.98 requests/s, 163.59 ms mean, 373.50 ms P95; concurrency 4 = 9.89 requests/s, 387.38 ms mean, 787.77 ms P95.

Concurrent load simulation (continued)

- With PyTorch intra-op and inter-op threads set to 1: concurrency 1 = 7.31 requests/s, 136.66 ms mean, 395.15 ms P95; concurrency 2 = 14.37 requests/s, 136.70 ms mean, 347.70 ms P95; concurrency 4 = 12.66 requests/s, 283.48 ms mean, 559.21 ms P95.

On this machine, thread limiting improved throughput under concurrency 2 and 4, but made single-request latency slower. At concurrency 4, cache misses rose to 196 across the three rounds versus 153 expected for 51 unique embeddings per round. This is consistent with duplicate encodes when concurrent requests miss the same LRU key. Recheck thread settings and cache behavior on deployment hardware.

Horizontal scaling strategy

- Create a stateless FastAPI scoring service and place replicas behind a load balancer. Sticky sessions are unnecessary because scoring state is not stored in a user session.
- Load the pinned model once during worker startup and report readiness only after it is available. Keep model files local to the image or a read-only shared volume to avoid startup downloads.
- Start with one inference worker per container. Scale replicas using CPU utilization, queue depth, and P95 latency; set memory headroom because every process has its own model and cache.
- Keep small resume requests synchronous. Put large batches on a bounded queue and return a job identifier so long batches do not occupy API workers.
- Retain the local bounded LRU cache initially. Add shared caching only if production metrics show repeated cross-replica misses. Monitor latency by stage, throughput, errors, process memory, model startup, and cache hit/miss counts.

Limitations and next checkpoints

The repository currently has no API route or deployment manifests, so HTTP latency, load-balancer behavior, and multi-replica scaling remain unmeasured. The load sample is small and local. Before production use, add the API boundary, run a larger HTTP load test across replicas, and investigate the observed score drift and duplicate cache-miss behavior under concurrency.